

同济大学

扫雷大作业实验报告

2451951 大数据 于沛可

2026/1/17

1. 题目

本次作业要求实现一个完整的扫雷演示与游戏系统，包含以下九个功能模块：

1.1 基础数组显示模块(1-2)

模块 1: 选择难度等级，随机生成地雷并显示内部数组

模块 2: 输入坐标，展开初始区域并显示

1.2 内部数组游戏模块(3-4)

模块 3: 基础版游戏,支持连续输入坐标进行游戏

模块 4: 完整版游戏，增加标记功能(&显示时间、@标记雷、#取消标记)

1.3 伪图形界面模块(5-7)

模块 5: 绘制中文边框框架并显示内部数据

模块 6: 检测鼠标位置并判断坐标合法性

模块 7: 鼠标点击一次展开初始区域

1.4 完整图形游戏模块(8-9)

模块 8: 支持左键打开、右键标记的基础图形游戏

模块 9: 完整游戏,显示剩余雷数和运行时间

2. 整体设计思路

2.1 程序架构

整个程序采用模块化设计，分为四个核心程序文件:

mine_sweeper_main.cpp	主控制模块：初始化程序=>显示菜单=>调用对应功能函数
mine_sweeper_menu.cpp	菜单显示模块
mine_sweeper_base.cpp	基础逻辑（地雷生成、数字计算、区域展开、胜负判断）实现
mine_sweeper_graph.cpp	图形界面实现（边框绘制、鼠标事件、格子高亮）(5-9)
mine_sweeper_tools.cpp	辅助工具函数（时间计算、坐标显示等）模块
mine_sweeper.h	头文件(函数声明、常量定义)

2.2 底层数据设计

本程序使用三个核心数组模拟扫雷状态:

char board[MAX_ROWS+2][MAX_COLS+2];	真实棋盘('0'-'8'表示雷数,'*'表示雷)
char display[MAX_ROWS+2][MAX_COLS+2];	显示棋盘('X'表示未打开)
char marked[MAX_ROWS+2][MAX_COLS+2];	标记数组('0'未标记,'1'已标记)

难度等级配置:

初级: 9×9, 10 个雷

中级: 16×16, 40 个雷

高级: 16×30, 99 个雷

2.3 关键设计思想

1. 参数化设计

函数通过参数控制不同行为，避免重复代码

例如: gameInit(rows, cols, mines, display, need_display_init)

need_display_init=false: 子题 1 不初始化 display

need_display_init=true：子题 2-4、8-9 初始化 display 为全 X

2. 状态同步

图形显示与内部数组严格同步，每次移动后立即更新：数组状态=>高亮格子=>显示数字

3. 分层设计

算法层：openArea()负责递归展开逻辑

显示层：printBoard()打印文字数组,drawFrame()绘制图形边框

交互层：screenToGrid()转换鼠标坐标,handleMouseMove()处理鼠标事件

3. 主要功能的实现

3.1 计算格子周围雷数

```
void calcNumbers(board, rows, cols):
    for i 从 1 到 rows:
        for j 从 1 到 cols:
            if board[i][j] == '0': // 只有不是雷的地方才需要计算数字
                count = 0
                for 偏移量 kr 从 -1 到 1:
                    for 偏移量 kc 从 -1 到 1:
                        if board[i + kr][j + kc] == '*':
                            count = count + 1
                board[i][j] = '0' + count // 将数字填入当前格子
```

3.2 递归展开空白区函数

```
void openArea(board, display, row, col, rows, cols):
    if 坐标(row, col) 越界:
        return
    if display[row][col] 已经不是'X' (说明已打开):
        return
    if board[row][col] 是雷 '*':
        return

    display[row][col] = board[row][col] // 打开当前格子

    if board[row][col] == '0': // 如果是空格，则递归检查周围8个方向
        for 偏移量 kr 从 -1 到 1:
            for 偏移量 kc 从 -1 到 1:
                if kr == 0 且 kc == 0: continue
                openArea(board, display, row + kr, col + kc, rows, cols)
```

3.3 屏幕坐标转换为棋盘下标

```
bool screenToGrid(MX, MY, rows, cols, &row, &col):
    // 获取当前难度下的格子宽高
    getCellSize(rows, cols, cell_width, cell_height)

    // 减去边框在控制台上的起始偏移量 (START_X=4, START_Y=3)
    // +2 和 +1 是为了跳过行标 A、B... 和列标 1、2... 的空间
    offsetX = MX - (4 + 2)
    offsetY = MY - (3 + 1)

    if offsetX < 0 或 offsetY < 0:
        return false // 点击在棋盘左上方以外

    // 计算下标：坐标除以格子间距
    col = offsetX / cell_width + 1
    row = offsetY / (cell_height + 1) + 1

    // 检查计算出的下标是否在合法棋盘范围内
    if row >= 1 且 row <= rows 且 col >= 1 且 col <= cols:
        return true
    return false
```

3.4 交互式布雷（addMines 改进版）

在 fun8/fun9 中，为了保证第一下点击不踩雷，使用了带安全区的布雷逻辑

```
void addMines(board, rows, cols, mines, safe_row, safe_col):
    added = 0
    while added < mines:
        r = 随机生成(1 到 rows)
        c = 随机生成(1 到 cols)

        // 检查 (r, c) 是否落在以 (safe_row, safe_col) 为中心的 3x3 安全区内
        in_safe_zone = false
        for 偏移 kr 从 -1 到 1:
            for 偏移 kc 从 -1 到 1:
                if r == safe_row + kr 且 c == safe_col + kc:
                    in_safe_zone = true

        if in_safe_zone:
            continue // 如果在安全区内，重新生成坐标

        if board[r][c] == '0': // 如果该处没有雷
            board[r][c] = '*'  // 放雷
            added = added + 1
```

3.5 格子高亮刷新

在 fun8/fun9 中，鼠标移动到某个格子时，需要变色反馈

```
void highlightCell(row, col, rows, cols, color):
    getCellSize(rows, cols, cell_width, cell_height)

    // 计算该格子内容区域在屏幕上的左上角起点
    // 算法基于边框偏移 + (下标-1)*格子间距
    startX = 4 + 2 + (col - 1) * cell_width
    startY = 3 + 1 + (row - 1) * (cell_height + 1)

    // 循环填充颜色，不打印字符，只改变背景色
    for i 从 0 到 cell_height - 1:
        cct_gotoxy(startX, startY + i) // 定位到格子的每一行
        设置颜色为 color
        打印 cell_width 长度的空格 // 用空格填充背景
```

3.6 绘制边框

```
void drawFrame(rows, cols):
    // 1. 获取当前等级对应的格子尺寸
    getCellSize(rows, cols, cell_width, cell_height)

    // 2. 打印列标 (1, 2, 3... a, b, c...)
    定位到 (START_X + 2, START_Y)
    for j 从 1 到 cols:
        if j <= 9: 打印数字 j
        else: 打印字符 ('a' + j - 10)
        向右偏移 cell_width 的距离

    // 3. 打印顶边框 (╔═══╤═══╗)
    定位到起点坐标
    打印 "╔"
    for j 从 1 到 cols:
        打印 "═══" (根据 cell_width 决定长度)
        if j < cols: 打印 "╤"
        else: 打印 "╗"

    // 4. 循环打印每一行及其底部分隔线
    for i 从 1 到 rows:
        // A. 打印行标 (A, B, C...) 和左边框
        在这一行开头打印 ('A' + i - 1)
        打印 "║"

        // B. 为这一行的每一个格子填充初始颜色 (黄色背景)
        for j 从 1 到 cols:
            highlightCell(i, j, rows, cols, COLOR_HYELLOW)
            if j < cols: 打印内部竖线 "│"
            else: 打印右边框 "║"

        // C. 打印行与行之间的横向分隔线 (╟───┼───╢)
        if i < rows:
            打印左接头 "╟"
            for j 从 1 到 cols:
                打印 "───"
                if j < cols: 打印 "┼"
                else: 打印 "╢"
        else:
            // D. 最后一行打印底边框 (╚═══╧═══╝)
            打印 "╚"
            for j 从 1 到 cols:
                打印 "═══"
                if j < cols: 打印 "╧"
                else: 打印 "╝"
```

3.7 fun8/9

```
void fun8_or_9():
    level = 初始化游戏参数(rows, cols, mines)
    初始化玩家界面 display 为全 'X'
    初始化标记数组 marked 为全 '0'
    绘制静态边框(rows, cols)
    开启鼠标交互模式()

    // 第一阶段：等待玩家首次点击
    while(1):
        读取 鼠标动作(MX, MY, MAction) 或 键盘按键(key)
        if MAction == 鼠标移动:
            悬停格子高亮处理(MX, MY)
        else if MAction == 左键单击:
            if 坐标有效(MX, MY):
                first_row, first_col = 屏幕坐标转棋盘坐标(MX, MY)
                start_tick = 获取系统当前时间() // 仅 fun9
                break // 进入第二阶段

    // 第二阶段：生成数据并开始游戏
    生成雷区 board (避开首次点击坐标)
    计算数字 calcNumbers(board)
    在框架内展开起始区域 openAreaInFrame(board, display, first_row, first_col)

    // 第三阶段：游戏主循环
    while(1):
        if 检查是否胜利():
            提示 "成功" 并显示用时, break

        读取 鼠标动作(MX, MY, MAction) 或 键盘按键(key)

        if MAction == 鼠标移动:
            悬停格子高亮处理(MX, MY)
        else if MAction == 左键单击:
            r, c = 屏幕坐标转棋盘坐标(MX, MY)
            if display[r][c] == 'X' 且 未被标记:
                if board[r][c] == '*':
                    高亮雷区, 提示 "踩雷失败", break
                else:
                    在框架内展开区域 openAreaInFrame(board, display, r, c)
        else if MAction == 右键单击:
            r, c = 屏幕坐标转棋盘坐标(MX, MY)
            if display[r][c] == 'X':
                切换标记状态 marked[r][c] ('0' <-> '1')
                更新剩余雷数显示, 在格子绘制 'F' 或 恢复掩盖
        else if key == 空格键:
            显示当前已用时间 // 仅 fun9
        else if key == ESC:
            break
```

4. 调试过程碰到的问题

4.1 边框第一列比其他列宽的问题

绘制边框时,第一列比其他列多出一个═的宽度,导致整个框架不对称,看起来像梯形。原因分析是交叉点╪ ╤覆盖时没有正确对齐,导致第一列实际宽度多了 1 个═。解决：我在画边框时进行对第一列的特判处理，同时所有交叉点╪统一向左偏移 1 个字符。

4.2 多重定义链接错误(LNK2005)

编译时出现链接错误:

mine_sweeper_graph.obj : error LNK2005: "struct GameTime __cdecl convertTime(long)" (?convertTime@@YA?AUGameTime@@J@Z) 已经在 mine_sweeper_base.obj 中定义

因为我起初在头文件 mine_sweeper.h 中直接实现了 convertTime 和 printTime 函数，每个包含该头文件的.cpp 文件都会有一份这两个函数的定义，链接时就冲突了。

4.3 缓冲区设置大小问题

起初我在 fun7 8 9 中没有加入 prepareConsoleForBoard 函数，导致屏幕会随着用户在菜单页滚轮缩放大小的改变而改变，从而导致显示棋盘时出现混乱的情况，我加入 prepareConsoleForBoard 函数，通过固定显示棋盘页面的大小，选择大棋盘页面也不会再出错。

5. 心得体会

1. 模块化设计的重要性

通过将程序分为base、graph、tools、main四个模块，每个模块职责明确，极大地提高了代码的可维护性。特别是当需要修改某个功能时，只需要关注对应的模块,不会影响其他部分。例如，当我需要修复边框对齐问题时，只需要修改graph.cpp中的drawFrame函数，不会影响base.cpp中的游戏逻辑。这种模块化的设计让我在后期调试时节省了大量时间。

2. 递归算法的优雅与实用

区域展开功能使用递归实现非常简洁优雅，核心逻辑不到20行。相比用栈或队列实现的非递归版本,递归代码更易理解和维护。但在调试时也发现，递归深度可能很大。好在扫雷游戏的棋盘大小有限(最大16×30)，即使最坏情况(整个棋盘都是0)递归深度也不会超过480层，不会导致栈溢出。这让我深刻理解了递归的适用场景。

3. 图形界面的细节处理

伪图形界面充满了细节:

中文字符占用2个字符宽度
交叉点的对齐需要向左偏移1个字符
第一列和其他列的填充逻辑不同
鼠标坐标转换需要考虑横线占用的高度

这些细节如果不仔细处理，就会出现对齐错误、黑色缝隙、误触发等问题。我通过画草图标注每个位置的坐标，最终才解决了所有对齐问题。对于复杂的布局问题，先设计后实现（在纸上画图）比边写边调效率高得多。

6. 附件：源程序

```
void fun9()
{
    int rows, cols, mines;
    char display[MAX_ROWS + 2][MAX_COLS + 2];
    char marked[MAX_ROWS + 2][MAX_COLS + 2];
    int marked_count = 0;
    long start_tick = 0;
    bool game_started = false;
    int level = gameInit(rows, cols, mines, display, true);
    prepareConsoleForBoard(rows, cols);
```

```
    cct_cls();
    cct_setcolor(COLOR_BLACK, COLOR_WHITE);
    cout << "等级为" << level << "（左键打开，右键标记，空格显示时间，ESC 退出）" << endl;
    for (int i = 0; i <= MAX_ROWS + 1; i++)
    {
        for (int j = 0; j <= MAX_COLS + 1; j++)
        {
            display[i][j] = 'X';
            marked[i][j] = '0';
        }
    }
    drawFrame(rows, cols);
    cct_setcursor(CCT_CURSOR_INVISIBLE);
    cct_enable_mouse();
    int MX, MY, MAction, key1, key2;
    int first_row = -1, first_col = -1;
    int last_highlight_row = -1, last_highlight_col = -1;
    int CELL_WIDTH, CELL_HEIGHT;
    getCellSize(rows, cols, CELL_WIDTH, CELL_HEIGHT);
    const int START_Y = 3;
    int info_y = START_Y + 1 + rows * (CELL_HEIGHT + 1) + 2;
    cct_setcolor(COLOR_BLACK, COLOR_WHITE);
    cct_gotoxy(0, info_y);
    cout << "剩余雷数：" << (mines - marked_count);
    cct_gotoxy(0, info_y + 1);
    cout << "[当前鼠标] 未移动";
    cct_gotoxy(0, info_y + 2);
    cout << "（提示：左键打开，右键标记，空格显示时间）";
    // 等待首次点击
    while (1)
    {
        int ret = cct_read_keyboard_and_mouse(MX, MY, MAction, key1, key2);
        if (ret == CCT_MOUSE_EVENT)
        {
            if (MAction == MOUSE_ONLY_MOVED)
            {
                int r, c;
                if (screenToGrid(MX, MY, rows, cols, r, c))
                {
                    if (r != last_highlight_row || c != last_highlight_col)
                    {
                        if (last_highlight_row > 0 && last_highlight_col > 0)
                        {
                            if (display[last_highlight_row][last_highlight_col] == 'X')
                            {
                                if (marked[last_highlight_row][last_highlight_col] == '1')
                                    highlightCell(last_highlight_row, last_highlight_col,rows, cols, COLOR_HRED);
                                else
                                    highlightCell(last_highlight_row, last_highlight_col,rows, cols, COLOR_HYELLOW);
                            }
                        }
                        if (display[r][c] == 'X')
                        {
                            if (marked[r][c] == '1')
                                highlightCell(r, c, rows, cols, COLOR_RED);
                            else
                                highlightCell(r, c, rows, cols, COLOR_YELLOW);
                        }
                        last_highlight_row = r;
                        last_highlight_col = c;
```

```
                }
                cct_setcolor(COLOR_BLACK, COLOR_WHITE);
                cct_gotoxy(0, info_y + 1);
                cout << "[当前鼠标] ";
                printCoordLabel(r, c);
                cout << "          ";
            }
            else
            {
                cct_setcolor(COLOR_BLACK, COLOR_WHITE);
                cct_gotoxy(0, info_y + 1);
                cout << "[当前鼠标] 位置非法
";
            }
        }
        else if (MAction == MOUSE_LEFT_BUTTON_CLICK)
        {
            int r, c;
            if (screenToGrid(MX, MY, rows, cols, r, c))
            {
                first_row = r;
                first_col = c;
                start_tick = clock();
                game_started = true;
                break;
            }
        }
    }
    else if (ret == CCT_KEYBOARD_EVENT && key1 == 27)
    {
        cct_disable_mouse();
        cct_setcursor(CCT_CURSOR_VISIBLE_NORMAL);
        cct_setcolor(COLOR_BLACK, COLOR_WHITE);
        cct_gotoxy(0, info_y + 4);
        to_be_continued(NULL);
        return;
    }
}
char board[MAX_ROWS + 2][MAX_COLS + 2];
initBoard(board, rows, cols);
addMines(board, rows, cols, mines, first_row, first_col);
calcNumbers(board, rows, cols);
openAreaInFrame(board, display, first_row, first_col, rows,
cols);
// 游戏主循环
while (1)
{
    bool win = true;
    for (int i = 1; i <= rows; i++)
    {
        for (int j = 1; j <= cols; j++)
        {
            if (board[i][j] != '*' && display[i][j] == 'X')
            {
                win = false;
                break;
            }
        }
        if (!win)
            break;
    }
    if (win)
    {
        struct GameTime gt = convertTime(start_tick);
        cct_setcolor(COLOR_BLACK, COLOR_WHITE);
        cct_gotoxy(0, info_y + 3);
        cout << "游戏成功, 一共用时 ";
        printTime(gt);
        cout << "秒" << endl;
        break;
    }
    int ret = cct_read_keyboard_and_mouse(MX, MY, MAction,
```

```
key1, key2);
        if (ret == CCT_MOUSE_EVENT)
        {
            if (MAction == MOUSE_ONLY_MOVED)
            {
                int r, c;
                if (screenToGrid(MX, MY, rows, cols, r, c))
                {
                    if (r != last_highlight_row || c != last_highlight_col)
                    {
                        if (last_highlight_row > 0 && last_highlight_col > 0)
                        {
                            if (display[last_highlight_row][last_highlight_col] == 'X')
                            {
                                if (marked[last_highlight_row][last_highlight_col] == '1')
                                {
                                    highlightCell(last_highlight_row, last_highlight_col,rows, cols, COLOR_HRED);
                                    showCellValue(last_highlight_row, last_highlight_col,'F', rows, cols);
                                }
                                else
                                    highlightCell(last_highlight_row, last_highlight_col,rows, cols, COLOR_HYELLOW);
                            }
                            else
                            {
                                highlightCell(last_highlight_row, last_highlight_col,rows, cols, COLOR_HWHITE);
                                showCellValue(last_highlight_row, last_highlight_col,display[last_highlight_row][last_highlight_col],rows, cols);
                            }
                        }
                        if (display[r][c] == 'X')
                        {
                            if (marked[r][c] == '1')
                            {
                                highlightCell(r, c, rows, cols, COLOR_RED);
                                showCellValue(r, c, 'F', rows, cols);
                            }
                            else
                                highlightCell(r, c, rows, cols, COLOR_YELLOW);
                            last_highlight_row = r;
                            last_highlight_col = c;
                        }
                        cct_setcolor(COLOR_BLACK, COLOR_WHITE);
                        cct_gotoxy(0, info_y + 1);
                        cout << "[当前鼠标] " << (char)('A' + r - 1) << "行";
                        if (c <= 9)
                            cout << c;
                        else
                            cout << (char)('a' + c - 10);
                        cout << "列          ";
                    }
                    else
                    {
                        cct_setcolor(COLOR_BLACK, COLOR_WHITE);
```

```
                cct_gotoxy(0, info_y + 1);
                cout << "[当前鼠标] 位置非法
";
            }
        }
        else if (MAction == MOUSE_LEFT_BUTTON_CLICK)
        {
            int r, c;
            if (screenToGrid(MX, MY, rows, cols, r, c) &&
display[r][c] == 'X' && marked[r][c] != '1')
            {
                if (board[r][c] == '*')
                {
                    struct GameTime gt =
convertTime(start_tick);
                    highlightCell(r, c, rows, cols,
COLOR_HRED);
                    showCellValue(r, c, '*', rows, cols);
                    cct_setcolor(COLOR_BLACK,
COLOR_WHITE);
                    cct_gotoxy(0, info_y + 3);
                    cout<<"你输了，游戏结束，一共用时 ";
                    printTime(gt);
                    cout << " 秒" << endl;
                    break;
                }
                openAreaInFrame(board, display, r, c, rows,
cols);
            }
        }
        else if (MAction == MOUSE_RIGHT_BUTTON_CLICK)
        {
            int r, c;
            if (screenToGrid(MX, MY, rows, cols, r, c) &&
display[r][c] == 'X')
            {
                if (marked[r][c] == '0')
                {
                    marked[r][c] = '1';
                    marked_count++;
                    highlightCell(r, c, rows, cols,
COLOR_HRED);
                    showCellValue(r, c, 'F', rows, cols);
                }
                else
                {
                    marked[r][c] = '0';
                    marked_count--;
                    highlightCell(r, c, rows, cols,
COLOR_HYELLOW);
                }
                cct_setcolor(COLOR_BLACK, COLOR_WHITE);
                cct_gotoxy(0, info_y);
                cout << "剩余雷数：" << (mines -
marked_count) << "  ";
            }
        }
    }
    else if (ret == CCT_KEYBOARD_EVENT)
    {
        if (key1 == ' ')
        {
            struct GameTime gt = convertTime(start_tick);
            cct_setcolor(COLOR_BLACK, COLOR_WHITE);
            cct_gotoxy(0, info_y + 3);
            cout << "已用时：";
            printTime(gt);
            cout << " 秒" << endl;
        }
        else if (key1 == 27)
        {
            break;
```

```
            }
        }
    }
    cct_disable_mouse();
    cct_setcursor(CCT_CURSOR_VISIBLE_NORMAL);
    cct_setcolor(COLOR_BLACK, COLOR_WHITE);
    cct_gotoxy(0, info_y + 5);
    to_be_continued(NULL);
}
```

说明：fun8 与 fun9 大体相同，只是去掉了显示时间的逻辑部分